

Collections Framework Data Structures

DATA TYPES AND EXCEPTIONS IN JAVA



Jim White
Java Instructor

What's a framework

- A framework is a set of reusable types providing generic functionality
 - Example of types: classes and interfaces
 - Enhances application's quality
 - Avoid having to create everything from scratch
- Frameworks are delivered in a package or packages in Java
- Frameworks come with guidelines and rules for use



¹ Photo by Ashkan Forouzani on Unsplash

Java Types

Two categories of **types** in Java:

- **Primitive types**
 - byte, short, int, long, float, double, char, boolean
- **Reference types**
 - Classes
 - Interface
 - Enums
 - Arrays
 - ...

The Collections Framework

- **Collections Framework:** Java's built-in set of generic data structure types
 - A package of types to store and manipulate **a group of objects**
- Defined in the `java.util` package
 - We must use import to use the framework types

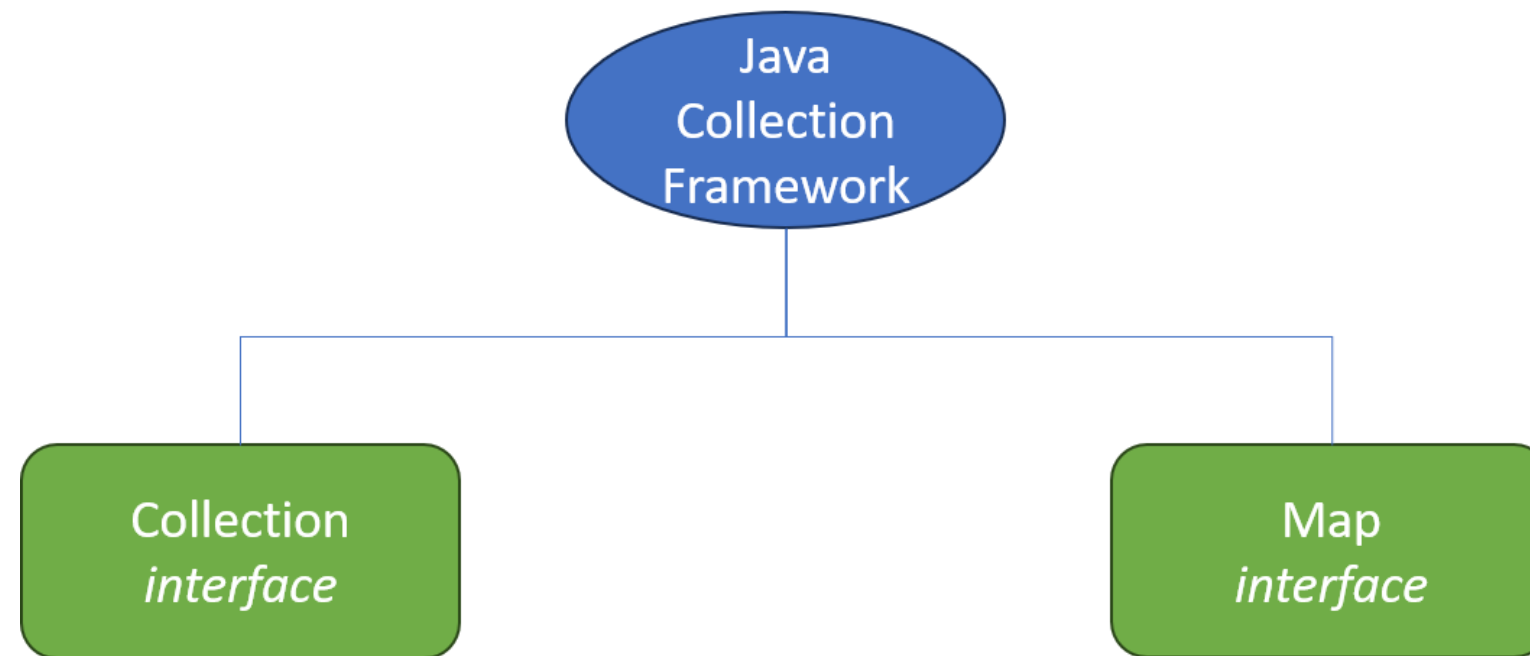
What about arrays?

- Java arrays can hold a group of objects or primitives
 - Collections Framework offers an alternative set of data structures
- Both have pluses and minus

Arrays	Collections
Not resizable	Dynamically sized (grow and shrink)
Stores primitives or objects	Stores only objects
Homogeneous - elements must be the same	heterogeneous - objects can be different
Special notation to access <code>[]</code> elements	Uses methods to access objects
Special syntax for initialization <code>{}</code>	Use new (no special initialization syntax)

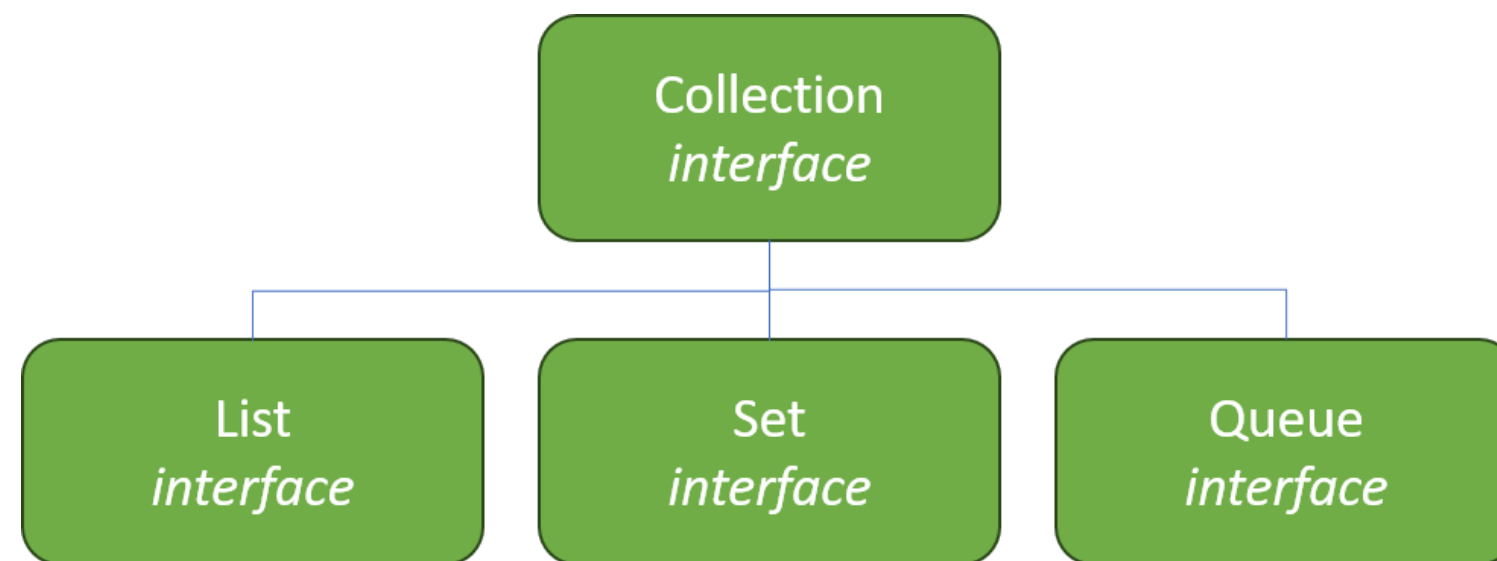
Collections Framework Types

- Collections Framework are divided into two basic types: `Collection` and `Map`
 - Represented by two interfaces: `java.util.Collection` & `java.util.Map`
 - Both have several implementing classes
- `import java.util.*;` to use any of the Collections Framework types



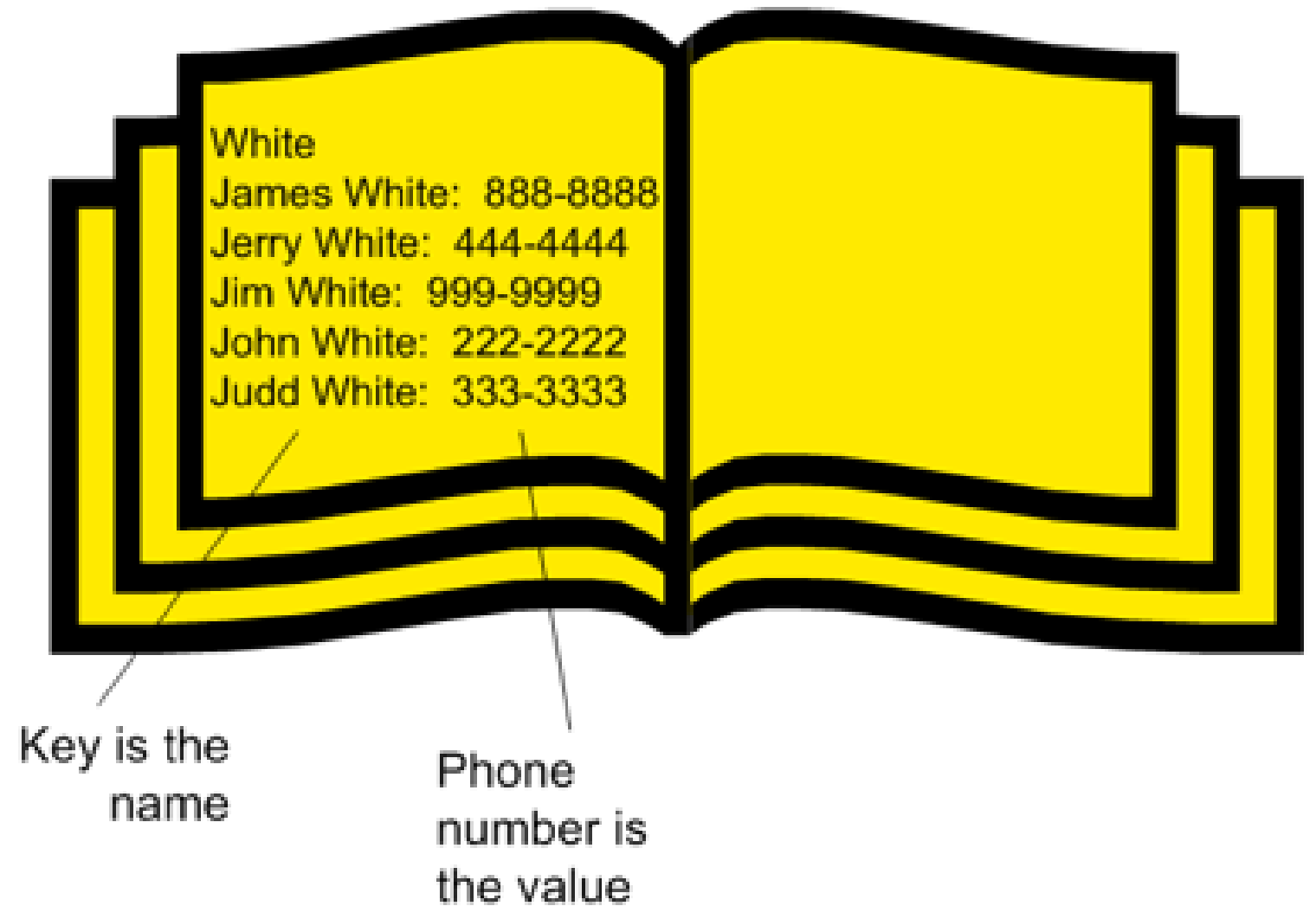
Collection

- `Collection` defines many kinds of groupings of objects
 - These are defined by additional interfaces
- Ordered group of objects: `List`
- Unordered group of objects: `Set`
- First-in-first-out group of objects: `Queue`
 - Operates like a line at a bank or ticket booth
- `.add(Object)` and `.remove(Object)` methods to modify any `Collection`



Map

- Map data structures behave like a phone book or dictionary
 - "key" objects -> to "value" objects
 - In phone book, a name -> phone number
 - In dictionary, a word -> word's definition
- Use the key object to modify a Map
 - `.put(Object key, Object value)` to add to a Map
 - `.remove(Object key)` to remove from a Map
- Many types of Map - like `HashMap`



Generics

- Collections and Maps are "parameterized"
 - Java generics are used to specify the objects in a `Collection` or `Map`
 - Generic syntax: `<Class>` with a parameterized type.
 - `< >` called the diamond operator

```
ArrayList<String> list = new ArrayList<String>(); // Construct with generics
```

Generics and type safety

- Generics allows for better type safety
 - Restricts objects allowed in the contents of `Collection` and `Map`

```
ArrayList<String> list = new ArrayList<String>();  
list.add("hello"); // Adding a String is ok  
list.add(new Integer(5)); // Trying to add an Integer causes compiler error
```

Not using generics

```
ArrayList list2 = new ArrayList(); // Legal but non-specific  
  
list2.add("hello");                // Now any type of object can be added  
list2.add(new Integer(5));
```

Variable declaration and assignment

- Variable declaration and assignment can be done separately

```
ArrayList<String> list = new ArrayList<String>(); //Single line declaration and assignment
```

```
ArrayList<String> list2; // Variable declaration...
```

```
list2 = new ArrayList<String>(); //... and assignment can be 2 statements
```

Let's practice!

DATA TYPES AND EXCEPTIONS IN JAVA

Lists

DATA TYPES AND EXCEPTIONS IN JAVA



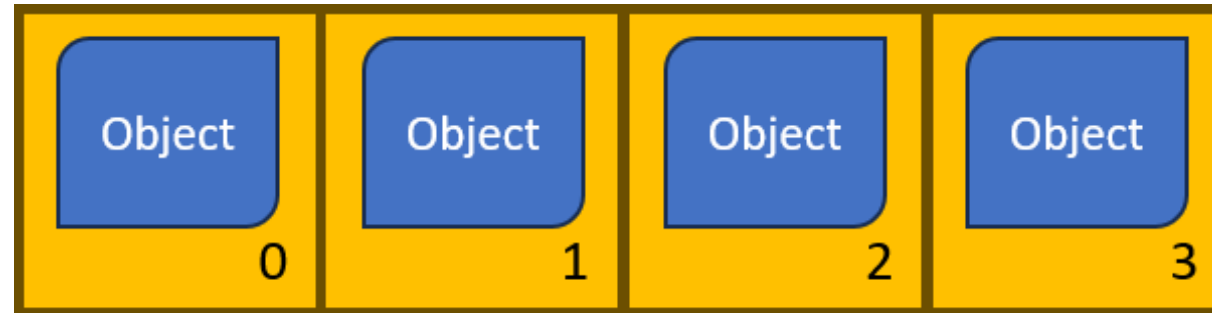
Jim White
Java Developer

List interface

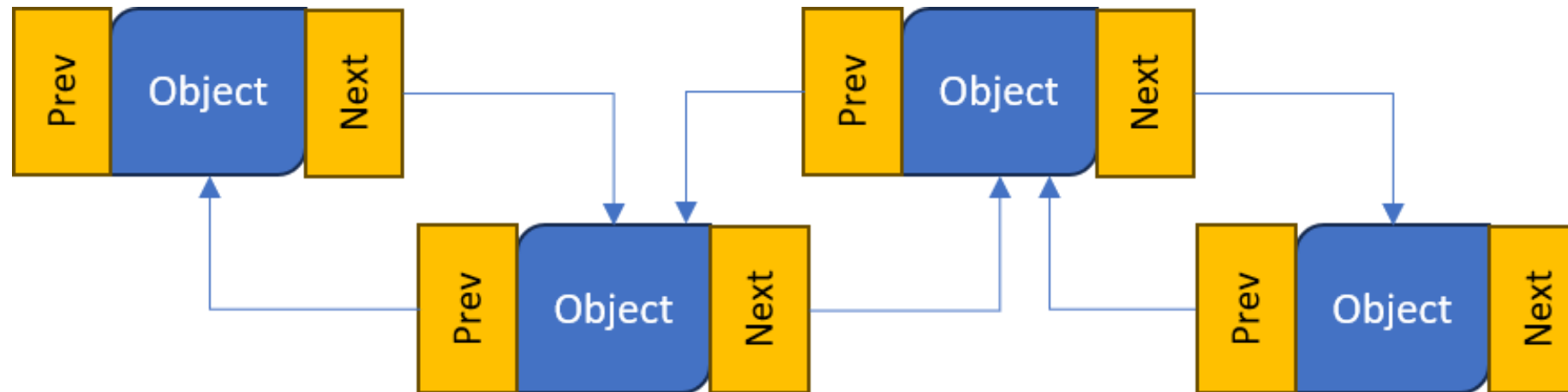
- `List` interface, sub-interface of `Collection`
 - Defines an **ordered** group of objects
 - Can contain duplicated objects
- Several implementations of `List`
 - All are similar in behavior
 - Implementation determines how objects are managed under the covers

List implementations

- `ArrayList` - a resizable, indexed (like arrays), ordered list of objects



- `LinkedList` - objects connected by links to the next and previous objects.



- No matter the implementation, add, remove, change objects in the same way

¹ See <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/List.html> for List types and details

ArrayList

- Construct an instance with `new` and use generics to specify content type
- Add an object to the list end with `.add(object)`
- Access an object with `.get(index)`
 - `ArrayList` has zero-based index
- Replace/change an object with `.set(index, object)`
- Remove an object with `.remove(index)`
- Clear or empty all objects with `.clear()`
- Count of objects with `.size()`

```
import java.util.ArrayList;
...
ArrayList<String> animals
    = new ArrayList<String>();
animals.add("horse");
animals.add("cow");
animals.add("horse"); // Duplicates allowed
String c = animals.get(1); // c="cow"
animals.set(1, "chicken");
// Removes the first horse
animals.remove(0);
// Removes all objects
animals.clear();
animals.size();
```

Objects and primitives

- Use wrappers to add a primitive to any Collections Framework object
- Primitives added are "autoboxed"
 - **Autobox:** process of automatically putting primitive in a wrapper

```
ArrayList<Integer> list = new ArrayList<Integer>();  
list.add(new Integer(5));  
list.add(6); // 6 is automatically wrapped by Integer then added
```

- Autoboxing works with all Collections Framework objects

Heterogeneous collections

- Collections Framework data structures can be *heterogenous*
 - Heterogenous = hold objects of different types
 - Use `Object` as the parameterized type

```
ArrayList<Object> list = new ArrayList<Object>(); // Allow any object
list.add(new Integer(5)); // Add an Integer
list.add(6); // Add an Integer using autoboxing
list.add("Hello"); // Add a String
```

Looping a List

- Use the "for each" syntax to loop through the objects of a `List`

```
ArrayList<String> animals = new ArrayList<String>();  
animals.add("horse");  
animals.add("cow");  
animals.add("chicken");  
  
for (String animal : animals) { // fore-each to loop through all in the ArrayList  
    System.out.println(animal);  
}
```

```
horse  
cow  
chicken
```

List println

- The contents of a `List` instance (like `ArrayList`) can be displayed with `println`

```
ArrayList<String> animals = new ArrayList<String>();  
animals.add("horse");  
animals.add("cow");  
animals.add("chicken");  
System.out.println(animals); // Display all the elements in the ArrayList
```

```
ArrayList<Object> list = new ArrayList<Object>();  
list.add(5);  
list.add("Hello");  
System.out.println(list); // Display all the elements in the ArrayList
```

```
[horse, cow, chicken]  
[5, Hello]
```

LinkedList

- `LinkedList` are constructed like `ArrayList` instances
- `LinkedList` have the same methods as `ArrayList`

```
import java.util.LinkedList;

...
LinkedList<String> cars // Create a new ...
    = new LinkedList<String>(); // ... LinkedList
```

```
cars.add("Ford"); // Add an object to the list
cars.add("Mercedes");
String c = cars.get(1);
cars.set(1, "Toyota"); // Replace an object
System.out.println(cars); // Display the list
cars.remove(0); // Remove an object
cars.clear(); // Remove all objects
cars.size(); // Get the list length
```

```
[Ford, Toyota]
```

Additional LinkedList methods

- Some additional methods
 - `addFirst()` add to start of the list
 - `addLast()` add to the end of the list
 - `removeFirst()` remove from start
 - `removeLast()` remove from end

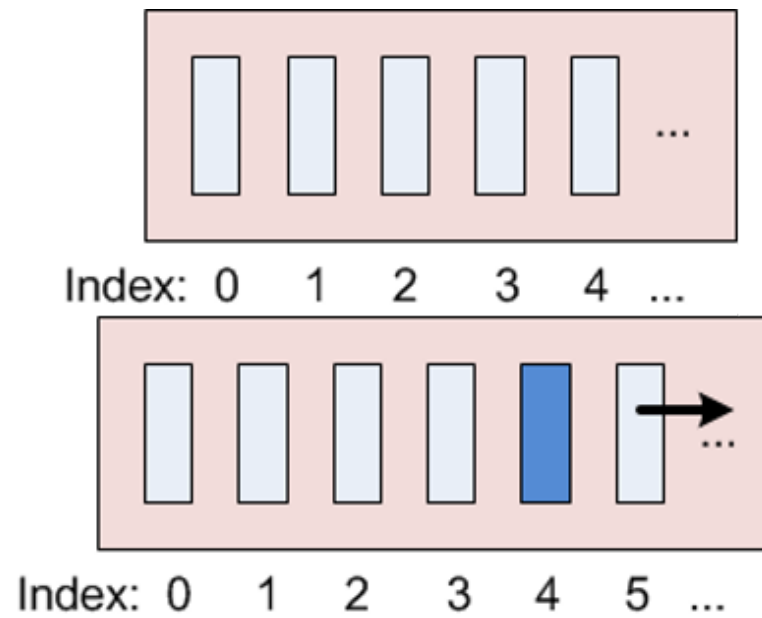
```
cars.addFirst("Fiat"); // Add to the beginning
cars.addLast("BMW"); // Add to the end
cars.removeFirst(); // Remove the first object
cars.removeLast(); // Remove the last object
```

Collection type similarities

- `ArrayList` and `LinkedList` look the same
 - Other types in Collections Framework will also have similarities
 - This is on purpose
- They share an interface: `List`
 - An example of polymorphism ("many-forms")

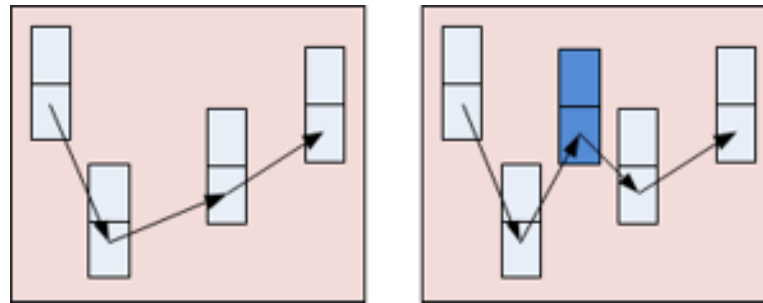
ArrayList vs LinkedList

- Operations on `ArrayList` and `LinkedList` (or any `List`) are the same
 - Pick of `List` type depends on use
- `ArrayList` considerations:
 - Fast at random access (ex: `list.get(11)`)
 - Slower at adding and removing elements in the middle
 - Because objects must physically moved when inserting/removing objects



ArrayList vs LinkedList

- `LinkedList` considerations:
 - Slower at random access (ex: `list.get(11)`)
 - Faster at adding and removing elements in the middle.
 - Because no objects are physically moved, just a couple of references changed



Let's practice!

DATA TYPES AND EXCEPTIONS IN JAVA

Sets, Queues

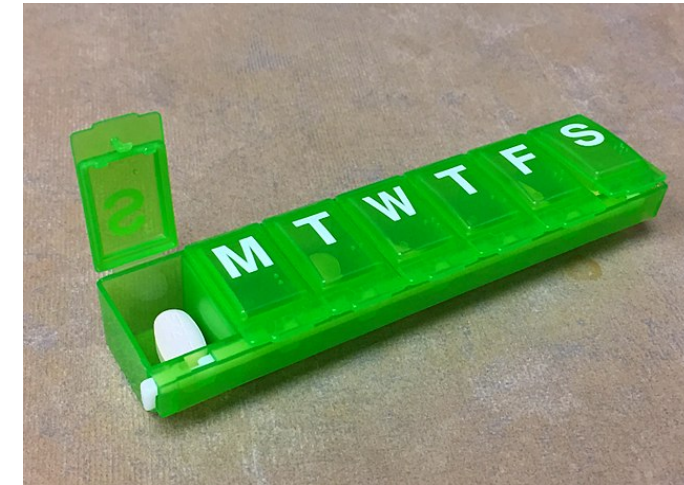
DATA TYPES AND EXCEPTIONS IN JAVA



Jim White
Java Developer

Set interface

- **Set** also a kind of **Collection**
 - Does not allow duplicate objects
 - Guarantees each element is unique
 - **Set** objects are, generally, unordered (therefore have no index)
- **List** s are like a pill box for objects; each object in a specific spot in the box
- **Set** s are like a sack holding objects; where objects are randomly held in the sack



¹ Images courtesy Wikimedia Commons

List vs Set

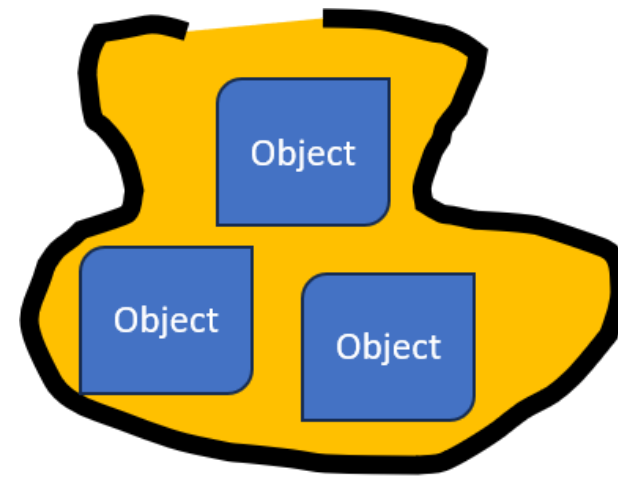
- When to use a Set
 - Guarantees uniqueness of its elements
 - Makes them faster for lookups
 - Checking if an object is in a Set
 - More memory efficient the larger the collection grows
 - Good for membership testing in caches
 - Example: finding unique words in a document

List vs Set

- When to use a List
 - Preserves order of elements and allows for duplicates
 - Perform better when accessing elements by position
 - Perform better when adding/deleting elements by index
 - Good where order of elements is important
 - Ex: managing steps of a sequence or songs in a playlist

Set implementation

- Many implementations of `Set`
 - `HashSet` a popular implementation
- `HashSet`
 - An unordered bag of objects
 - Faster than other `Set` implementation for insert, delete and lookup searches
 - Generally use more memory than other `Set` implementations
 - Allow storing a single `null`



HashSet construction

- Use generic parameterized constructor to create a `HashSet`

```
HashSet<String> set = new HashSet<String>();
```

- `HashSet` is found in `java.util` package
 - Requires an import
 - `import java.util.HashSet`

HashSet methods

- Use `.add()` and `.remove()` to add, remove objects
- Use `.remove()` and then `.add()` to replace an object
- Use `.contains()` to check if object already exists
- Duplicates are ignored
- Allows `null`
- Ordered not guaranteed

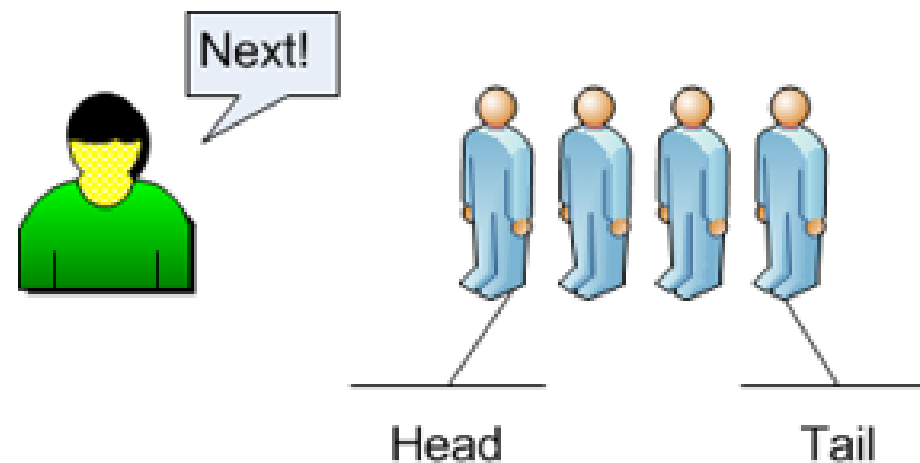
```
set.add("France");  
set.add("Japan");  
set.add("Brazil");  
set.add("Egypt");  
set.add(null); // null is allowed  
set.remove("Brazil");
```

```
boolean z =  
    set.contains("France"); // z is true  
set.add("Japan"); // Ignored  
System.out.println(set);
```

```
[null, Japan, Egypt, France]
```

Queue interface

- `Queue` data structure processes objects in a first in, first out (FIFO) order
 - First object added is first removed
 - Like a line at a ticket booth
 - Has a head and tail object



- Several implementations of `Queue`
 - Similar in behavior and operations

ArrayBlockingQueue

- Popular `Queue` implementation
 - An array under the covers
- **Attention:** `ArrayBlockingQueue` is in `java.util.concurrent` package
 - Not in `java.util` package

ArrayBlockingQueue construction

- Use generic parameterized constructor to create a `ArrayBlockingQueue`
 - Has a capacity or object count limit specified with the constructor

```
import java.util.concurrent; // At the top of the class

// Create new queue that can store 4 Strings
ArrayBlockingQueue<String> queue = new ArrayBlockingQueue<String>(4);
```

ArrayBlockingQueue methods

- Use `.add(object)` or `.offer(object)` to add to the tail
 - `.add(object)` throws exception when `Queue` is at capacity
 - `.offer(object)` ignores the new object when at capacity

```
ArrayBlockingQueue<String> queue
    = new ArrayBlockingQueue<String>(4);
queue.offer("France");
queue.offer("Japan");
queue.offer("Brazil");
queue.offer("Egypt");
queue.offer("China"); // Ignores China

// Causes IllegalStateException
// queue.add("China");

System.out.println(queue);
```

```
[France, Japan, Brazil, Egypt]
```

ArrayBlockingQueue methods

- Use `.remove()` or `.poll()` to remove from the head
 - `.remove()` throws an exception when `Queue` is empty
 - `.poll()` returns `null` when `Queue` is empty
- Does not allow null

```
ArrayBlockingQueue<String> queue
    = new ArrayBlockingQueue<String>(4);
String x = queue.poll(); // x is null
// Causes NoSuchElementException
// String y = queue.remove();

queue.offer("France");
String next = queue.poll();
System.out.println(next);
```

France

Let's practice!

DATA TYPES AND EXCEPTIONS IN JAVA

Maps

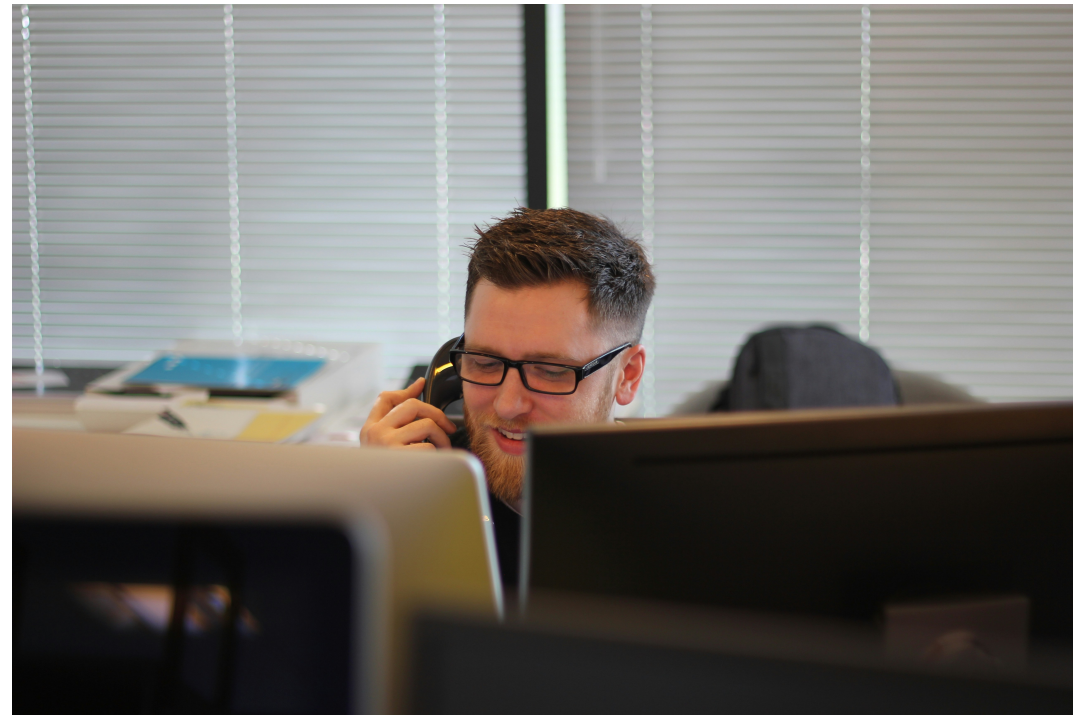
DATA TYPES AND EXCEPTIONS IN JAVA



Jim White
Java Developer

Maps are lookup tables

- Maps serve as lookup table data structures
 - Store a "value" at a specific "key"
 - Use the "key" to retrieve the stored "value"
- Example: calling a doctor
 - They lookup a health record (the *value* object) given a name/birth date (the *key*)

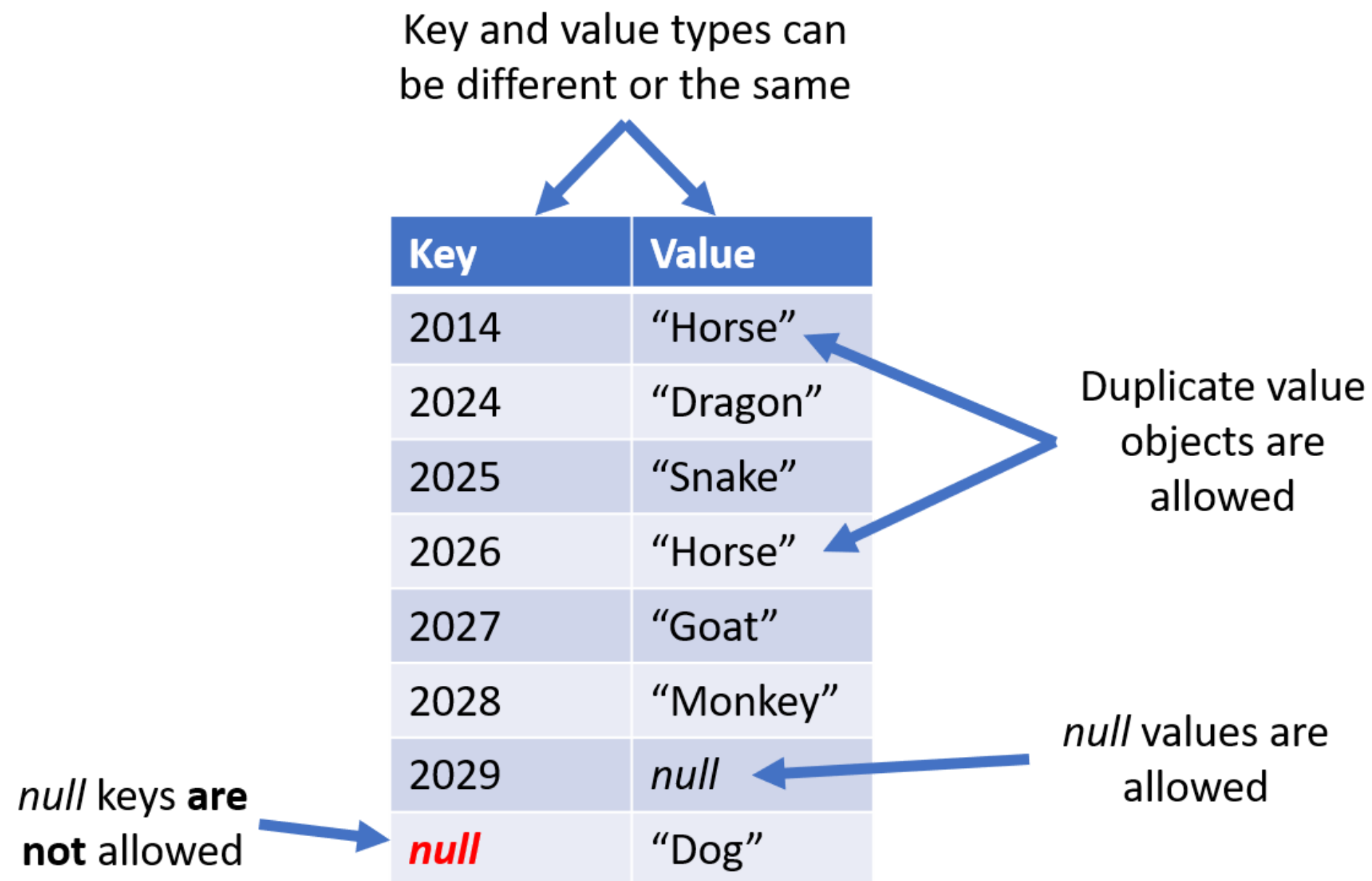


Map interface

- `Map` interface defines operations on the key and value pairs
 - How to store or "put" a value into the store at a specific key
 - How to "remove" the value at the key from the `Map`
 - How to retrieve the value given a key
- Several implementations of `Map` that are similar in behavior
 - A popular implementations of `Map` is `HashMap`

¹ See <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Map.html> for details

HashMap keys and values



HashMap construction

```
HashMap<Integer, String> map = new HashMap<Integer, String>();
```

- The parameterized/ generic constructor for a `HashMap` requires **two types**
 - The first type = the *key* type

`HashMap<Integer, String>`

- The second type = the *value* type

`HashMap<Integer, String>`

Key (Integer type)	Value (String type)
0	"Jim"
1	"James"

- Requires import of `java.util.HashMap`

HashMap methods

- Use `.put(key, value)` to add a key/value pairs to the table
- Use `.remove(key)` to remove the key/value pair specified at the key
- Use `.get(key)` to retrieve the value from the table at the key

```
import java.util.HashMap

...
HashMap<Integer, String> map
    = new HashMap<Integer, String>();
map.put(0, "Jim");
map.put(1, "James");
map.put(3, null);
map.put(4, "James");
map.remove(0);
System.out.println(map);
```

```
{1=James, 3=null, 4=James}
```

```
String nickname = map.get(1);
System.out.println(nickname);
```

```
James
```

Collections

- `java.util.Collections` is a supporting class
- Used to aid in sorting, filling, copying, searching, and more
 - Only has `static` methods

¹ See <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Collections.html>

Collection methods

- Add many objects to a `List`
 - `addAll(List list, Object a, b, ...)`
- Return a count of an object in a collection
 - `frequency(Collection c, Object o)`
- Reorder the objects in a `List`
 - `reverse(List list)`
- Sort the objects in a `List`
 - `sort(List list)`
- Replace all the objects in a `List` with another object
 - `fill(List a, Object o)`

```
ArrayList<String> x = new ArrayList<String>();
Collections.addAll(x,
    "milk", "bread", "eggs", "milk");
System.out.println(x);
int cnt = Collections.frequency(x, "milk");
System.out.println(cnt);
Collections.reverse(x);
System.out.println(x);
Collections.sort(x);
System.out.println(x);
Collections.fill(x, "sugar");
System.out.println(x);
```

```
[milk, bread, eggs, milk]
2
[milk, eggs, bread, milk]
[bread, eggs, milk, milk]
[sugar, sugar, sugar, sugar]
```

Arrays

- Sometimes need to convert an array (e.g., `int[]`) to a `List`
 - Java arrays are not resizable and we may need to grow/shrink the elements
 - `List` has greater capability (searching, sorting, etc.)
- `java.util.Arrays` is another supporting class
 - Allows converting a Java array to a `List`

Arrays example

```
String[] arrayCountries = {"France", "Japan", "Brazil", "Egypt", "China"};  
List<String> countries = Arrays.asList(arrayCountries);  
System.out.println(countries);
```

```
[France, Japan, Brazil, Egypt, China]
```

Let's practice!

DATA TYPES AND EXCEPTIONS IN JAVA